

J · A · R · V · I · S

NEURAL CORE · DİJİTAL TEKNİSYEN ASİSTANI

Teknik Proje Dokümanı Tam Detay

Vizyondan çalışan betaya: mimari, gereksinimler, uygulanan modüller (güvenli araç katmanı, hafıza, kimlik, ses, canlı panel), Windows/WSL2 kurulumu ve yol haritası.

ÇALIŞAN BETA

147 TEST ✓

AGENT + TOOLS

PERMISSION LAYER

HİBRİT AI

NEURAL CORE UI

Sürüm

0.2.0

Tarih

22.08.2026

Durum

Beta ·
WSL2

Donanım

Ryzen 7 5800X · RTX 3080 Ti 12GB ·
32GB

Doküman Haritası

01	Proje Vizyonu ve Kapsam	Ne olduğu, ana amaç, yetenekler, kapsam felsefesi
02	Donanım Gerçeği ve Kararlar	RTX 3080 Ti 12 GB, VirtualBox, D1-D8 karar özeti
03	Sistem Mimarisi ve Fizibilite	Model stratejisi, RAG, teşhis, güvenlik, yol haritası
04	Gereksinim Denetimi	Eksikler, çelişkiler, karar kaydı
05	Uygulanan Modüller	Çekirdek, güvenlik, hafıza, kimlik, ses, canlı panel
06	Arayüz Tasarımı — Neural Core	Yatay kontrol paneli ve durum sistemi
07	Kurulum: Windows + WSL2	Seçilen yol, adımlar, doğrulama, .env
08	Yol Haritası, Maliyet ve Riskler	Fazlar, maliyet kalemleri, risk önlemleri

Proje Vizyonu ve Kapsam

J.A.R.V.I.S., basit bir sohbet botu değil; sesli iletişim kurabilen, görsel analiz yapabilen, bilgisayar donanım ve yazılımını çözümleyebilen, araç kullanabilen, hafızası olan ve teknik servis süreçlerini takip edebilen modüler bir yapay zekâ asistanıdır. Ana amaç, kullanıcının bilgisayar teknik servis çalışmalarında gerçek bir dijital teknisyen yardımcısı olmaktır.

Temel Çalışma Döngüsü

J.A.R.V.I.S.'in özü tek bir prensipte toplanır:

Bilgi topla → analiz et → olası nedenleri belirle → test öner → sonucu değerlendir → kullanıcıya anlaşılır şekilde yardımcı ol.

J.A.R.V.I.S. kullanıcının yerine karar veren kontrolsüz bir sistem değildir; kullanıcı sistemin ana yöneticisidir. Kritik işlemler açık onay olmadan yapılmaz.

Hedeflenen Yetenekler

- Sesli iletişim (STT → LLM → ElevenLabs TTS) ve doğal dil anlayışı
- iPhone kamerasından görsel analiz; bilgisayar bileşeni tanıma
- Host sistem telemetrisi: CPU/GPU sıcaklık, RAM, disk/SMART
- Güvenli araç kullanımı: terminal, dosya, uygulama — izin katmanı ardında
- Arıza teşhis motoru (Diagnostic Brain): belirtiden olasılık sıralı hipoteze
- RAG teknik bilgi tabanı ve katmanlı uzun süreli hafıza
- Servis vakası kaydı, otomatik servis raporu, ajanda ve proaktif uyarılar
- Neural Core görsel kimliği ve teknik kontrol merkezi paneli

Kapsam Felsefesi

En büyük risk teknoloji değil, kapsamdır. Bu yüzden J.A.R.V.I.S. tek seferde dev bir uygulama olarak değil, her biri bağımsız test edilebilen modüller halinde, fazlı olarak geliştirilir. Analiz → Mimari → Modül Tasarımı → Kod → Test → Entegrasyon sırası izlenir.

Donanım Gerçeği ve Kararlar

Bu bölüm donanımın gerçek sınırlarını ve kod öncesi verilen kritik kararları özetler. Ayrıntılı gerekçeler 3. ve 4. bölümdedir.

Donanım Gerçeği

Bileşen	Değer	J.A.R.V.I.S. için anlamı
CPU	Ryzen 7 5800X (8C/16T)	Embedding, ses ön-işleme, RAG için fazlasıyla yeterli
GPU	Zotac RTX 3080 Ti · 12 GB VRAM	En sıkı kısıt — ama 12 GB 14B modeli rahat alır (~9 GB)
RAM	32 GB	Vektör DB, embedding, uygulama katmanı için bol
VirtualBox video	256 MB (emüle)	İlgisiz. Gerçek VRAM ile karıştırılmamalı

VirtualBox + GPU çalışmaz. VirtualBox NVIDIA CUDA'yı guest'e geçirmez; pratik PCIe passthrough yoktur. Yerel AI için **bare-metal Linux** veya **WSL2** gerekir. Ayrıntı §6.

Verilen Kararlar

#	Karar	Sonuç
D1	Teşhis hedefi	Kendi makinem (host). Sistem araçları çalıştığı makineyi okur; ayrı/açılmayan müşteri makinesi karmaşası V1'de yok. ✓ Karar verildi
D2	Ağ modeli	Ev LAN ile başla; saha için Tailscale/WireGuard (açık)
D3	Veri sınıflandırma	Müşteri/kişisel veri yerel; genel sorgu bulut olabilir (açık)
D4	CRITICAL onay	İkinci kanal + yazılı doğrulama cümlesi ✓ V1'de uygulandı
D5–D8	Bildirim, bütçe, yetki, saklama	Öneriler hazır; ilgili fazlarda netleşecek

Sistem Mimarisi ve Fizibilite

J.A.R.V.I.S. — Mimari Analiz ve Fizibilite Raporu

Durum: **Analiz / Mimari onayı bekleniyor.** Bu doküman kod içermez. Amaç: Projeyi eleştirmek, eksikleri bulmak, donanımı hesaplamak ve onaylanabilir bir V1 mimarisi önermek.

Bu rapor seni memnun etmek için değil, projeyi **gerçekten çalışır** hale getirmek için yazıldı. Bazı hedeflerin bugünkü donanımına ya da genel olarak gerçekçi değil; bunları açıkça işaretledim ve her eleştirinin yanına uygulanabilir bir alternatif koydum.

0. Tek Cümlelik Özet

Projenin **%80'i tamamen gerçekçi ve değerli**; ama üç noktada beklentin gerçeklikle çarpışıyor: **(1)** VirtualBox üzerinde RTX 3080 Ti ile ciddi AI hesabı yapmak mümkün değil, **(2)** telefon kamerasından anakart model/soket tanıma "Iron Man" seviyesinde güvenilir olmayacak, **(3)** "her şeyi yapan" tek bir sistemi baştan kurmaya çalışmak projenin en büyük başarısızlık riskidir. Doğru yol: **dar ama sağlam bir V1**, hibrit (yerel + bulut) mimari ve modüler bir çekirdek.

1. Proje Gerçekçi mi?

Kısmen — ama parçalara ayrılırsa evet.

Yetenek	Gerçekçilik	Not
Sesli iletişim (STT→LLM→TTS)	✓ Yüksek	Olgun teknoloji, ElevenLabs + Whisper çalışır
Doğal dil / reasoning	✓ Yüksek	Hibrit (yerel + bulut LLM) ile
Sistem bilgisi / donanım okuma (sıcaklık, disk, SMART)	✓ Yüksek	Linux araçlarıyla kolay
Terminal / dosya / uygulama kontrolü (izin katmanıyla)	✓ Yüksek	Güvenlik katmanı şart
RAG teknik bilgi tabanı	✓ Yüksek	Standart mimari
Hafıza (konuşma, vaka, ajanda)	✓ Yüksek	SQLite + vektör DB
Ajanda / proaktif bildirim	✓ Yüksek	Event-driven
iPhone mikrofon/hoparlör/ses	✓ Orta-Yüksek	Custom app + WebRTC ile
Görsel analiz — genel ("bu bir anakart")	● Orta	Bulut vision ile iyi, yerel ile idare eder
Görsel analiz — tam model/soket/hata LED'i tanıma	● Düşük	Güvenilir değil. Aşağıda detaylı.
Arıza teşhis motoru (teknisyen gibi)	● Orta	Saf LLM yetmez; hibrit motor gerekir
"Sürekli dinleyen" wake word	● Orta	iOS'ta arka plan kısıtları ciddi engel
Akıllı gözlük / AR	● Uzun vade	V1 için değil, soyutlama ile hazırlanır

Sonuç: Proje bir "hafta sonu projesi" değil; bu **1-2 yıllık ciddi bir mühendislik yolculuğu**. Ama modüler kurarsan her ay çalışan bir şeyin olur.

2. En Büyük Teknik Zorluklar (Öncelik Sırasıyla)

- Kapsam (scope) yönetimi.** Bu projenin bir numaralı riski teknoloji değil, **her şeyi aynı anda yapmaya çalışmak**. 26 yetenek var; 3'ünü mükemmel yapmak, 26'sını yarım yapmaktan iyidir.
- VirtualBox + GPU.** Mevcut kurulumun AI hesabı için **kullanılamaz** (\$6). Bu çözülmeden yerel model konusu havada kalır.
- Görsel donanım tanıma güvenilirliği.** En çok heyecanlandığın özellik, en az güvenilir olanı (\$9).
- Ses gecikmesi (latency).** Doğal his için tüm zincirin streaming olması gerekir; her adım gecikme ekler (\$19).
- Arıza teşhisinin güvenilirliği.** LLM "halüsinasyon" yapar; teknik serviste yanlış teşhis pahalıdır. Yapılandırılmış motor şart (\$20).
- iOS arka plan kısıtları.** Apple, sürekli mikrofon dinleme ve arka plan ağ bağlantısını agresif şekilde kısıtlar (\$16, \$5).
- Güvenlik.** LLM'e sistem erişimi vermek = kendine bir "içeriden tehdit" yaratmak. İzin katmanı opsiyonel değil (\$12).

3–5. Donanım Yeterliliği: RTX 3080 Ti · 5800X · 32 GB

Doğrulanmış donanım (22.08.2026 güncellemesi). Önceki sürümde 5900X / RTX 3080 10 GB / 24 GB yazıyordu; gerçek yapılandırma aşağıdadır. GPU farkı önemli: **3080 Ti = 12 GB**, 3080'in 10 GB'ı değil.

Bileşen	Model	Not
CPU	AMD Ryzen 7 5800X	8 çekirdek / 16 iş parçacığı, Zen 3
GPU	Zotac RTX 3080 Ti	12 GB GDDR6X , GA102
RAM	32 GB	Sistem toplamı

RTX 3080 Ti — 12 GB Bir Eşiği Aşıyor

GPU'nun gücü işlemcide değil, **VRAM'inde** ve bu hâlâ en sıkı kısıtın — ama 12 GB, 10 GB'dan anlamlı biçimde daha rahat:

- **12 GB VRAM.** VirtualBox'taki 256 MB "video belleği" ile **hiçbir alakası yoktur** — o emüle edilmiş 2D masaüstü belleğidir, CUDA'da kullanılmaz. Bu ikisini asla karıştırma.
- **Kritik kazanım: 14B modeller artık rahat sığıyor.** 10 GB'da 14B "sıkışık, başka hiçbir şeye yer yok" durumundaydı; 12 GB'da ~3 GB boşluk kalıyor. Günlük model 7B değil **14B** olabilir.

12 GB VRAM'e gerçekçi olarak sığanlar:

Görev	Gerçekçi seçim	VRAM
Yerel LLM (günlük)	Qwen2.5 14B (Q4_K_M)	~9.0 GB
Yerel LLM (hafif)	Qwen2.5 7B / Llama 3.1 8B (Q4)	~4.7–4.9 GB
STT	faster-whisper large-v3 (int8)	~2–3 GB
Yerel Vision	Qwen2-VL-7B / MiniCPM-V (Q4)	~6–8 GB
Embedding	bge-m3 / e5	<1 GB

Neyin aynı anda çalışabildiği (asıl mesele bu):

Kombinasyon	Toplam	Durum
7B LLM + Whisper large-v3	~7.7 GB	✅ Rahat — sesli asistan için ideal
14B LLM tek başına	~9.0 GB	✅ Rahat
14B LLM + Whisper	~12 GB	🔴 Sınırdan/taşar
14B + Whisper + Vision	~18 GB	❌ Sığmaz

Sonuç değişmiyor ama yumuşuyor: Hepsini aynı anda çalıştıramazsın, yani **hibrit mimari hâlâ zorunlu**. Ama artık iki net kullanım profiline var: - **Sesli mod:** 7B + Whisper birlikte, GPU'da rahat. - **Derin analiz modu:** 14B tek başına, STT/vision o an buluta veya sıraya.

Model router'ın işi tam olarak bu geçişi yönetmek.

32 GB RAM — Rahat

- 32 GB, planladığımız 24 GB'dan fazla. Vektör DB, embedding, uygulama katmanı ve model yükleme tamponları için bol alan var.
- **CPU+GPU hibrit inference** (llama.cpp katman offload) mümkün ama **yavaştır** — CPU'ya taşan her katman token/sn'yi düşürür. Sesli, gerçek zamanlı kullanım için modeli tamamen VRAM'e sığdır; taşıma. 12 GB ile 14B'de buna zaten gerek kalmıyor.
- **5800X (8 çekirdek / 16 iş parçacığı)** — embedding, ses ön-işleme ve RAG için fazlasıyla yeterli. 5900X'e göre 4 çekirdek az ama bu iş yükünde darboğaz değil; asıl yük GPU'da.

Karar: Donanım, doğru kurulumla iyi bir V1 için **yeterli ve önceki varsayımdan bir tık daha iyi**. Tek gerçek engel yazılım tarafında: **VirtualBox**.

6. VirtualBox Problemi (Kritik — Önce Bunu Çöz)

Net cevap: VirtualBox üzerinde RTX 3080 Ti'yi CUDA/AI için kullanamazsın.

- VirtualBox **NVIDIA CUDA'yı guest'e geçirmez**. Sunduğu grafik (VBoxVGA, VMSVGA) emüle edilmiş bir adaptördür; `nvidia-smi` guest içinde GPU'yu göremez, CUDA çalışmaz.
- VirtualBox'ta **pratik bir PCIe passthrough yoktur**. (DeneySEL/kararsız "PCI passthrough" özelliği modern NVIDIA GPU'larla çalışmaz.)
- 256 MB, gerçek 12 GB VRAM ile ilgisizdir.

Seçeneklerin Karşılaştırması

Yöntem	GPU/CUDA	Zorluk	Öneri
Yöntem	GPU/CUDA	Sensörler	Maliyet
---	---	---	---
VirtualBox	✗ Yok	✗ Yok	—
WSL2 (Windows üzerinde)	✓ Tam	✗ Yok	Sıfır
Bare-metal Linux	✓ Tam (en hızlı)	✓ Var	Yeni SSD + kurulum
KVM/QEMU + VFIO	✓ Tam, host GPU'yu kaybeder	✓ Var	Yüksek karmaşıklık

Karar: WSL2 (22.08.2026)

Başlangıçta bare-metal Linux öneriliyordu. Kullanıcının makinesinde **Windows zaten kurulu** olduğu ortaya çıkınca karar WSL2 lehine değişti:

- **CUDA çalışır**. NVIDIA, WSL2'de CUDA'yı resmen destekler; GPU, Windows sürücüsü üzerinden gelir ve 12 GB VRAM tam kullanılır.
- **Maliyet sıfır**. Yeni disk yok, bölümlleme yok, veri kaybı riski yok.
- **Windows kalır**. Teknik servis işi için gereken üretici araçları ve BIOS yazılımları erişilebilir kalıyor; yeniden başlatma gerekmiyor.
- **Kod değişmiyor**. Proje Linux üzerine yazıldı; WSL2 Linux.
- **Kurulum ~30 dakika**, bare-metal'de yarım gün.

Bilerek kabul edilen kayıp: WSL2 donanım sensörlerini vermez — CPU sıcaklığı ve fiziksel disk SMART verisi okunamaz (`nvidia-smi` istisnadır, çalışır). Bu, VirtualBox'ta da yoktu; yani gerileme değil, mevcut durumun üstüne GPU eklemek. Sensörler gerçekten gerekli çıkarsa bare-metal

seçeneği açık kalıyor — ama bu karar artık tahminle değil, **canlı kullanım deneyimiyle** verilecek.

WSL2'de en kritik kural: WSL içine NVIDIA sürücüsü kurulmaz. GPU Windows'taki sürücüden gelir; içeride ayrıca sürücü kurmak çalışan kurulumu bozar.

Aksiyon: VirtualBox yalnızca demo/gözlem için. Geliştirme ve gerçek kullanım **WSL2** üzerinde (docs/KURULUM-WSL2.md).

7. Önerilen Linux Mimarisi

- **Dağıtım:** Ubuntu 22.04/24.04 LTS (NVIDIA sürücü + CUDA toolkit desteği en sorunsuz olan).
- **NVIDIA yığını:** Proprietary driver + CUDA 12.x + cuDNN.
- **İzolasyon:** Servisleri **Docker Compose** ile ayır (LLM sunucusu, vektör DB, uygulama API'si, Redis). GPU için `nvidia-container-toolkit`.
- **Süreç yönetimi:** systemd veya Docker restart policy; proaktif izleme servisi için ayrı bir long-running process.
- **Neden konteyner:** §21'deki modüler yapıyla birebir uyumlu; her modülü bağımsız güncelleyip test edebilirsin.

8. Önerilen LLM Seçenekleri

Prensip: Tek model değil, görev-bazlı yönlendirme (model router).

Yerel (RTX 3080 Ti, 12 GB) — düşük gecikmeli / gizli veri için: - **Qwen2.5 14B Instruct** (Q4_K_M) günlük sürücü; hafif/sesli mod için **Qwen2.5 7B** veya **Llama 3.1 8B**. Türkçe için Qwen2.5 ve Llama 3.1 makul; küçük modeller Türkçe'de İngilizce kadar iyi değil — bunu test et. - Çalıştırma: başlangıçta **Ollama** (kolay), üretim/paralel istek için **llama.cpp server** veya tek kullanıcı senaryosunda yeterli.

Bulut — ağır reasoning / karmaşık teşhis / en iyi Türkçe için: - Büyük reasoning gerektiğinde bir bulut LLM'e (frontier model) çağrı yap. Hassas veriyi yerelde tut, genel/karmaşık soruları buluta gönder.

Neden hibrit: 12 GB VRAM ile yerel bir model "iyi" olur ama frontier bulut modellerin karmaşık teşhis kalitesine ulaşamaz. İkisini birleştir.

Servis motoru karşılaştırması:

Araç	Ne zaman
Ollama	★ Başlangıç, geliştirme, tek kullanıcı — en kolay
llama.cpp	İnce kontrol, GGUF quant, CPU offload esnekliği
vLLM	Yüksek throughput / çok kullanıcı — sende aşırı , 10 GB'a sığması zor
TGI	vLLM benzeri, kurumsal — gereksiz
Transformers	Deney/fine-tune, üretim servisi için değil

Öneri: V1 = **Ollama (yerel) + bir bulut LLM API'si (router ile).**

9. Önerilen Vision Seçenekleri — ve Dürüst Uyarı

Burada beklentini düşürmem gerekiyor. "Kamerayı anakarta tut, JARVIS tam modeli, soketi, VRM'i, hata LED'ini okusun" senaryosu bugünkü teknolojiyle **güvenilir çalışmaz**. Nedenleri:

- Anakart üstündeki model yazısı küçük, açılı, parlama/ışık altında; OCR hataları yüksek.
- Aynı görünümlü onlarca kart var; model varyantlarını görüntüden ayırmak insan uzman için bile zor.
- Vision modelleri **kendinden emin ama yanlış** cevap verir (halüsinasyon). Teknik serviste yanlış model bilgisi tehlikelidir.

Gerçekçi kapsam: -  "Bu bir ATX anakart", "bu bir GPU", "bu RAM slotu", "bu 24-pin güç konnektörü" gibi **genel bileşen tanıma** → iyi çalışır. -  Görünür bir **model/etiket yazısını OCR ile okuma** → ancak net, yakın, düz çekilmiş fotoğrafta. -  Fotoğraftan **kesin model/varyant/soket çıkarımı** → güvenilmez. Bunun yerine: kullanıcıya "etikete/model yazısına yakın çek" dedir, OCR ile metni al, sonra o metni **RAG + web arama** ile doğrula.

Model seçimi:

Seçenek	Kalite	Not
Bulut vision (frontier multimodal)	★ En iyi	OCR + akıl yürütme çok üstün; V1 için bunu kullan
Qwen2-VL 7B (yerel, Q4)	Orta	GPU'da yer kaplar, LLM ile çakışır
MiniCPM-V (yerel)	Orta	Hafif, idare eder

Öneri: Vision'ı **V1'de buluta ver** (kalite + VRAM tasarrufu). Gizlilik gerektiren görüntüler için sonradan yerel bir vision modeli ekle. Ve **JARVIS'e "emin değilim" dedirt** — yanlış kesinlikten iyidir.

10. Önerilen Speech-to-Text (STT)

Seçenek	Gecikme	Doğruluk	Türkçe	Donanım	Maliyet
faster-whisper large-v3 (yerel, GPU)	Orta-düşük	Yüksek	İyi	~2-3 GB VRAM	Ücretsiz (elektrik)
whisper.cpp (yerel, CPU/GPU)	Orta	Yüksek	İyi	Esnek	Ücretsiz
Deepgram / bulut streaming STT	★ Çok düşük	Yüksek	Değişken	Yok	Kullanım başı
OpenAI/ElevenLabs STT (bulut)	Düşük	Yüksek	İyi	Yok	Kullanım başı

Değerlendirme: - Türkçe + gizlilik + maliyetsizlik istiyorsan → **faster-whisper large-v3 (yerel)**. Ama GPU'yu LLM ile paylaşması gerekir. - En düşük gecikme + streaming istiyorsan → **bulut streaming STT**.

Öneri: V1 = **faster-whisper (yerel)** başla; gecikme çok yüksek gelirse sesli mod için bulut streaming STT'ye geç. VAD (voice activity detection) ekle ki sürekli değil, konuşma bittiğinde işlesin.

11. ElevenLabs Entegrasyonu

- ElevenLabs TTS kalitesi mükemmel; Türkçe destekli seslerini seç.
- **Streaming API kullan** (chunk chunk ses) — yoksa cümle bitmeden ses başlamaz, gecikme hissedilir.
- **Mimari not:** TTS'i doğrudan iPhone'a değil, **JARVIS Server üzerinden** akıt. Böylece API key sunucuda kalır (telefonda gömülü key = güvenlik açığı). Server ElevenLabs'ten alır → iPhone'a WebRTC/WebSocket ile iletir.
- **Maliyet uyarısı:** ElevenLabs karakter başı ücretlidir; çok konuşan bir asistan aylık faturayı şişirebilir. Kısa cevaplar (\$15 "gereksiz konuşmayan" kişiliği) hem UX hem maliyet için iyi.
- **Yedek plan:** Gizli/offline durumlar veya maliyet için yerel bir TTS (ör. Piper) fallback olarak eklenebilir.

12–15. iPhone Mimarisi + Kamera/Mikrofon Aktarımı + Protokol Karşılaştırması

Protokol Karşılaştırması

Protokol	Ses (çift yönlü)	Kamera/Video	Gecikme	Not
WebRTC	★ Mükemmel	★ Mükemmel	En düşük	Gerçek zamanlı ses+video için tasarlandı; NAT geçişi, jitter buffer dahili
WebSocket	● İyi (chunk)	● İdare eder	Düşük-orta	Kontrol mesajları + ses chunk'ları için ideal; basit
gRPC	● Streaming var	●	Düşük	Sunucu-sunucu için güçlü; mobil gerçek-zamanlı sesde WebRTC kadar iyi değil
HTTP (REST)	✗	✗	Yüksek	Sadece komut/kontrol, canlı akış için değil
Raw UDP	Teorik	Teorik	En düşük	Kendin RTP/jitter/NAT yazacaksın = WebRTC'i yeniden icat etmek
Raw TCP	✗	✗	Head-of-line blocking	Canlı ses için kötü

Öneri: - **Canlı ses (çift yönlü) ve kamera akışı → WebRTC.** Tekerleği yeniden icat etme; NAT, jitter, kayıp paket işini WebRTC hallediyor. - **Kontrol/komut/durum mesajları → WebSocket** (JSON). Basit ve yeterli. - **Pratik V1 kısayolu:** WebRTC ilk sürümde karmaşıksa, MVP'yi **WebSocket + ses chunk'ları (push-to-talk)** ile yap; canlı düşük gecikme kritikleşince WebRTC'e geç. Böylece iOS tarafında erken takılmazsın.

iOS Tarafı

- **Swift + SwiftUI** (arayüz), **AVFoundation** (mikrofon/hoparlör/kamera).
- Apple Developer Program gerekir (~\$99/yıl) — cihaza gerçek uygulama yüklemek ve arka plan yetenekleri için pratikte şart.
- **Gerçekçilik uyarısı:** iOS **arka planda sürekli mikrofon dinlemeyi ve uzun süreli arka plan ağ bağlantısını kısıtlar.** "Uygulama kapalıyken bile sürekli dinleyen JARVIS" App Store kurallarına ve OS kısıtlarına takılır. V1'de **uygulama açıkken / push-to-talk** ile başla.

Güvenli Eşleşme (Pairing)

- İlk eşleşmede telefon ↔ server arası **QR kod ile token değişimi** (server bir kez QR gösterir, telefon okur).
- Sonrasında **mutual TLS veya imzalı token (JWT)** ile her bağlantıda kimlik doğrula. Yerel ağda çalışıyorsa self-signed sertifika + pinning.
- API key'ler **asla** telefonda gömülü olmasın; hepsi server'da.

16. Agent Mimarisi

Saf "prompt at, cevap al" yetmez. J.A.R.V.I.S. bir **agent loop** olmalı:

Kullanıcı girdisi (ses/metin/görüntü)

- Orchestrator (context toplar: hafıza + RAG + sistem durumu)
- LLM (plan yapar, tool çağırır)
- Tool Manager (izin katmanından geçer, çalıştırır)
- Sonuç LLM'e döner (gözlem)
- LLM devam eder / cevabı üretir
- TTS → ses

- Orchestrator** merkezde: hangi modeli (router), hangi tool'u, hangi hafızayı çağıracağına karar verir.
- Başlangıçta bir agent framework (ör. yapılandırılmış tool-calling) kullanabilirsin ama **çekirdeği kendi kodunda tut** — framework'e fazla bağımlı olma; abstraction'ını koru (§21).

17. Tool Calling Mimarisi + Güvenli Soyutlama

Evet — LLM'in işletim sistemine doğrudan/sınırsız erişimi kesinlikle olmamalı. İstedğin katman doğru:

LLM

- Tool Manager (tool'ları şema ile tanımlar, girdi doğrular)
- Permission Layer (risk sınıfı + onay + audit log)
- OS / donanım / dosya sistemi

- Her tool **açık şema** ile tanımlı (ad, parametre, risk sınıfı).
- LLM ham shell'e erişmez; **whitelist edilmiş, doğrulanmış tool'lara** erişir.
`run_terminal_command` bile bir **allowlist + tehlikeli komut filtresi** arkasında olmalı.
- Her tool çağırısı **audit log**'a yazılır (ne, ne zaman, kim onayladı).

18. Memory Mimarisi

Katman	Ne saklar	DB önerisi
Conversation Memory	Son konuşmalar, kısa vadeli bağlam	Redis (ephemeral) + SQLite (kalıcı özet)
User Memory	Tercihler, kişisel profil	SQLite / Postgres
Technical Knowledge	Teknik bilgi, dokümanlar	Vektör DB (RAG)
Service Case Memory	Servis vakaları, teşhis geçmişi	Postgres (yapısal) + vektör DB (arama)
Task / Calendar Memory	Görev, randevu, hatırlatma	SQLite / Postgres

Karar: - **V1: SQLite** (tek dosya, sıfır kurulum) + **bir vektör DB** (RAG için). - **Büyükünce:** **PostgreSQL + pgvector** (yapısal + vektör tek yerde) + **Redis** (ephemeral konuşma bağlamı, pub/sub, proaktif event kuyruğu). - Redis'i erken zorlamana gerek yok; SQLite ile başla.

19. RAG Mimarisi

Doküman (PDF/manual/web/not)

- Yükleme & parse (PDF metin + tablo çıkarımı)
- Chunking (semantik, ~500–1000 token, %10–20 overlap)
- Embedding
- Vektör DB'ye yaz (metadata: kaynak, sayfa, cihaz tipi)

Sorgu → embed → benzerlik ara (+ metadata filtre) → top-k → LLM'e context

- **Embedding modeli:** bge-m3 veya çok dilli e5 — **Türkçe destekli ve çok dilli** olması önemli (dokümanların Türkçe/İngilizce karışık).
- **Chunking:** Sabit boyut değil; başlık/bölüm sınırlarına saygılı **semantik chunking**. Teknik manuel'lerde tablo ve adım listelerini bölme.
- **Reranking** ekle (retrieval sonrası) — teknik doğruluk için değerli.

Vektör DB karşılaştırması:

DB	Başlangıç	Uzun vade	Not
FAISS	✅ Hızlı	🟡	Kütüphane, sunucu değil; metadata/filtre zayıf
Chroma	★ En kolay	🟡	Prototip için harika, ölçekte sınırlı
Qdrant	🟡	★ En iyi	Filtreleme, kalıcılık, prod-ready
pgvector	🟡	✅	Zaten Postgres kullanıyorsan tek DB avantajı

Öneri: **V1'de Chroma** ile başla (hız), **6-12 ay içinde Qdrant'a** (veya Postgres'e geçtiysen pgvector'a) taşı. Soyutlama koy ki geçiş kolay olsun.

20. Teknik Servis Teşhis Motoru

En kritik mimari kararlardan biri. Saf LLM YETMEZ. LLM tek başına tutarsız ve halüsinasyonludur; teknik teşhiste bu pahalıya patlar.

Önerilen: Hibrit teşhis motoru.

Belirti → LLM (belirtiyi yapılandırılmış forma çevirir)

→ Decision Tree / Rule Engine (bilinen arıza ağaçları)

→ Veri toplama tool'ları (SMART, sıcaklık, loglar)

→ Hipotez listesi + olasılık (basit skorlama/gevşek Bayesian)

→ Bir sonraki en bilgilendirici test'i öner

→ Sonuç → hipotezleri güncelle → döngü

→ RAG (üretici doküman / troubleshooting guide) ile destekle

- **State machine + karar ağacı** çekirdek olsun (deterministik, izlenebilir).
- **LLM'i "sınırlı ve doğrulanan" bir akıl yürütücü** olarak kullan, tek karar verici olarak değil.
- **Bayesian tam gerekmez** başta; her hipoteze basit olasılık ağırlığı yeterli. İleride vaka verisi biriktikçe gerçek Bayesian'a geçebilirsin.
- Bilinen arıza senaryolarını (ör. "açılıyor görüntü yok") **yapılandırılmış playbook**'lar olarak sakla; LLM bunları çalıştırsın.

Özet: Saf LLM ❌ → **Karar ağacı + rule engine + LLM + RAG** ✅.

21. Güvenlik Mimarisi

Risk sınıflandırması doğru; şöyle uygula:

Sınıf	Örnek	Politika
LOW	Sistem bilgisi, sıcaklık okuma	Otomatik, log
MEDIUM	Dosya değiştirme, uygulama açma	Otomatik + log + geri alınabilir
HIGH	Sistem uyarı, servis durdurma	Kullanıcı onayı
CRITICAL	Disk format, partition sil, BIOS flash	Açık, iki-adımlı onay + doğrulama cümlesi

Permission Architecture: - Her tool bir risk sınıfıyla **kayıtlı**; LLM sınıfı değiştiremez. - **CRITICAL** için: JARVIS ne yapacağını açıkça söyler → kullanıcı sözlü/yazılı **açık onay** verir (ör. belirli bir cümleyi tekrarlar) → ancak sonra çalışır. - **Dry-run** modu: yıkıcı işlemler önce simüle edilip kullanıcıya gösterilsin. - **Audit log** her şey için; geri alma (undo) mümkün olan yerde. - **Sandbox:** terminal komutları allowlist + tehlikeli-pattern reddi (`rm -rf /` , `dd` , `mkfs` , `> /dev/sda` vb. asla otomatik değil).

22. Ajanda Sistemi

- **V1: Yerel takvim DB** (SQLite tablosu: görev, tarih, tür, hatırlatma, durum). CRUD + hatırlatma yeterli.
- **Proaktif entegrasyon:** Görevler §23'teki event sistemine bağlanır (yaklaşan teslim → bildirim).
- **İleride:** iOS EventKit (iPhone Calendar) / Google Calendar API ile **tek yönlü sonra çift yönlü** senkron. Bir **CalendarProvider abstraction** koy ki backend değişse çekirdek değişmesin.

23. Proaktif Sistem

Evet, event-driven mimari gerekli — ama gürültü kontrolüyle.

Sensörler (sıcaklık, SMART, disk doluluk, ajanda)

- periyodik/eşik-tetikli izleme servisi
- Event bus (Redis pub/sub veya basit kuyruk)
- Kural + eşik + debounce/cooldown
- Önem filtresi (sadece anlamlı olanlar)
- Bildirim (iPhone push / sesli uyarı)

- **Debounce/cooldown zorunlu:** aynı uyarıyı sürekli tekrarlama. "CPU 85°C" bir kez bildir, düşene kadar sus.
- **Önem eşiği:** sadece eyleme dönük uyarılar (SSD sağlığı düşüyor, disk %95 dolu, teslim yarın). Gereksiz bildirim asistanı sinir bozucu yapar.
- Başlangıçta **basit bir long-running izleme servisi** yeter; tam event bus'ı büyüyünce ekle.

24. Öğrenme: Fine-tuning Gerekli mi?

Net cevap: Hayır, V1 (ve muhtemelen V2) için fine-tuning gerekmez.

Senin tarif ettiğin döngü **zaten doğru**:

geri bildirim → vaka kaydı → değerlendirme → bilgi tabanı → gelecekte retrieval

Bu **RAG + memory + tools** ile "öğrenme" — ve senin ihtiyacın olan tam bu. Modelin ağırlıklarını değiştirmeden, **yeni bilgiyi bilgi tabanına yazarak** sistem "öğrenir." Kontrollü, geri alınabilir, güvenli.

- **Fine-tuning ne zaman?** Sadece **davranış/stil/persona** tutarlılığı için (JARVIS'in konuşma tarzı), **bilgi eklemek için değil**. Bilgi = RAG.
- **LoRA/QLoRA ne zaman?** İleride, elinde yüzlerce gerçek servis vakası birikince ve model belirli bir **format/tarzı** öğrenmesi gerekirse QLoRA (RTX 3080 Ti'de 7B-14B için mümkün). Ama bu bir **optimizasyon**, temel değil.
- **Uyarı:** Kontrolsüz otomatik öğrenme (kullanıcı dedi → ağırlık değişti) **istemediğin gibi tehlikelidir** — haklısın, onu yapma.

Karar: RAG + memory + tools yeterli. Fine-tuning'i yol haritasının **çok ilerisine** koy.

25. Yerel + Bulut Hibrit + Model Router

Yaklaşımın doğru. Model router gerekli.

Görev	Nereye
Basit komut / kısa cevap	Yerel LLM
Hassas/gizli veri	Yerel LLM
Büyük reasoning / karmaşık teşhis	Bulut LLM
Güncel araştırma	Web arama tool'u
Ses (TTS)	ElevenLabs
STT	Yerel Whisper (veya bulut streaming)
Vision (V1)	Bulut multimodal

- **Router** basit başlasın: görev tipi + gizlilik bayrağı + karmaşıklık tahminine göre yönlendir. Makine öğrenmesiyle karmaşıklıklaştırmaya; kural tabanlı router V1 için yeter.
- **Gizlilik kuralı:** kişisel/sistem verisi içeren istekler **asla otomatik buluta gitmesin** — kullanıcı bayrağı olmadan yerelde kalsın.

26. Gecikme (Latency) — Gerçekçi Hedefler

Tüm zincir streaming olmalı, yoksa doğal hissetmez:

Konuşma bitişi → STT (streaming) → LLM (streaming) → TTS (streaming) → ses

Gerçekçi hedefler (yerel + iyi kurulum): - İlk ses çıkışına kadar (time-to-first-audio): **~800 ms - 1.5 sn** iyi. - Tam bulut zincir + ağ: **1.5-3 sn** olabilir. - **<500 ms "insan gibi anlık"** hedefi bu donanım/mimari ile **gerçekçi değil** — bunu baştan kabul et.

Gecikmeyi düşürme teknikleri: - Streaming her katmanda (özellikle LLM ilk token + ElevenLabs streaming). - VAD ile konuşma bitişini hızlı algıla. - LLM cevabının **ilk cümlesini** TTS'e hemen gönder, gerisi akarken. - Kısa cevap kişiliği (§28) hem gecikme hem doğallık için iyi.

27. Akıllı Gözlük Yol Haritası

V1'e koyma — ama mimariyi ona hazır tut. Doğru içgüdü: çekirdek değişmeden istemci değişmeli.

Soyutlama: İstemciyi (iPhone / gözlük) çekirdekten **Client Abstraction** ile ayır:

Herhangi bir istemci (iPhone, gözlük, web)

→ standart I/O sözleşmesi: {audio_in, video_in, audio_out, display_out, events}

→ JARVIS Core (istemcinin ne olduğunu bilmez)

- Core sadece **ses/görüntü akışları ve olaylar** görür; cihazın iPhone mı gözlük mü olduğunu umursamaz.
- Gözlük geldiğinde sadece **yeni bir istemci adaptörü** yazarsın; Core aynı kalır.
- AR ekran (display_out) opsiyonel bir kanal olarak sözleşmede dursun, başta boş geç.

28. Kişilik

- Kişilik bir **sistem prompt + davranış kuralları** katmanı olarak tanımlanır (ayrı, versiyonlanabilir bir dosya).
- Kurallar: sakin, teknik, kısa, gereksiz konuşmayan, **emin olmadığında açıkça belirten** ("emin değilim, doğrulayayım"), bilinç/duygu iddia etmeyen.
- **Emin olmama** davranışı teknik doğruluk için kritik — özellikle vision ve teşhiste (\$9, \$20).

29. "Sürekli Dinliyor" / Wake Word

- **Wake-word teknolojisi:** cihaz-ıçi hafif modeller (ör. Porcupine gibi gömülü wake-word motorları) — düşük gecikme, düşük tüketim.
- **Ama iOS gerçeği:** Uygulama arka plandayken sürekli mikrofona erişimi Apple tarafından ciddi kısıtlıdır ve App Store politikalarına takılır. Cihaz-ıçi "Hey Siri" tarzı her zaman-açık dinleme **OS seviyesi ayrıcalık** ister, üçüncü parti uygulamalar için pratikte kapalı.
- **Gerçekçi V1:** Uygulama açıkken wake-word / **push-to-talk** (butona bas-konuş). "Kapalıyken bile sürekli dinleyen" senaryosunu **gözlük veya ayrı bir donanım cihazı** aşamasına ertele (kendi donanımında bu kısıt yok).
- Gizlilik: sürekli dinleme = sürekli veri; wake-word'ü **cihazda** tut, sadece tetiklenince buluta/servera gönder.

30. Test ve Evaluation Sistemi

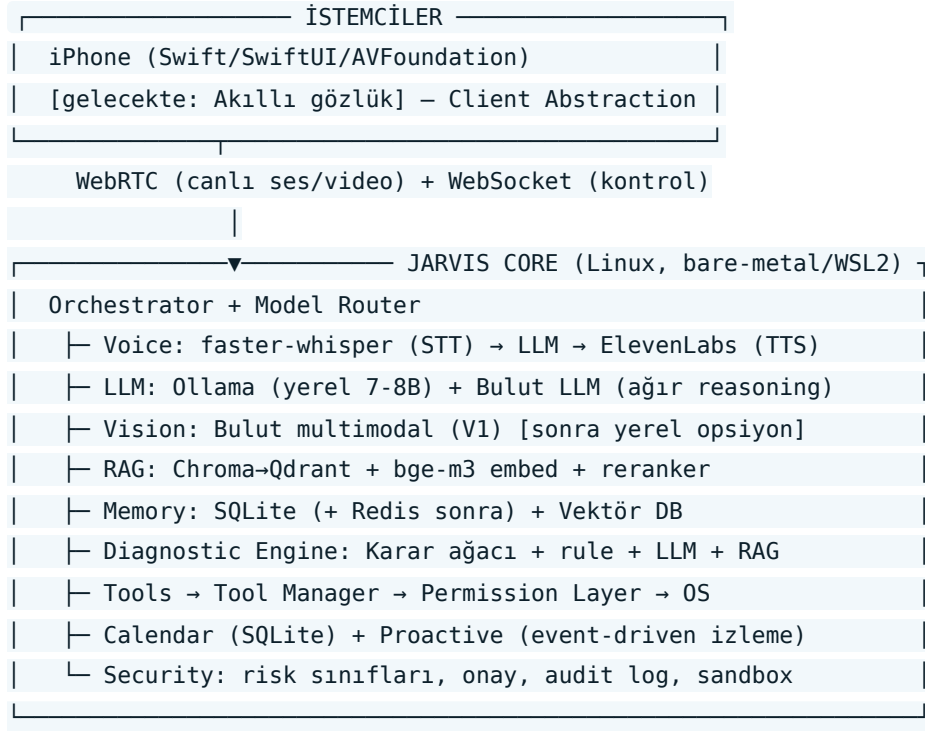
Ciddi proje için doğru talep. Katmanlar:

Kategori	Ne test eder	Yaklaşım
Unit	Fonksiyonlar, tool'lar	pytest
Integration	Modüller arası akış	pytest + test container'lar
Tool	Her tool doğru/güvenli çalışıyor mu	Mock OS + gerçek sandbox
LLM Eval	Cevap kalitesi/tutarlılık	Altın set + LLM-as-judge
Vision Eval	Bileşen tanıma doğruluğu	Etiketli fotoğraf seti
Speech Eval	STT WER, TTS anlaşılabilirlik	Türkçe ses seti, WER metriği
Memory Eval	Doğru hatırlama/getirme	Senaryo testleri
Security	İzin katmanı bypass edilemiyor mu	Kırmızı-takım testleri
Latency	Uçtan uca süre	Benchmark harness
Diagnostic	Teşhis doğruluğu	Gerçek servis vakaları benchmark'ı

En değerli varlığı: Kendi **gerçek servis vakalarından** oluşan bir benchmark. Her çözdüğün vakayı (belirti → teşhis → çözüm) kaydet; JARVIS'i bu sette ölç. Bu, projeni herhangi bir chatbottan ayıran şey olacak.

SONUÇ FORMATI

A) ÖNERİLEN MİMARİ



B) DONANIM GEREKSİNİMLERİ

- **Zotac RTX 3080 Ti (12 GB)** — bare-metal Linux üzerinde, CUDA ile. **VirtualBox terk edilecek.**
- 32 GB RAM rahat. Ryzen 7 5800X (8C/16T) bu iş yükü için yeterli.
- Tek seferde **tek ağır GPU işi** (LLM); STT/Vision router ile yönetilir veya buluta verilir.
- iPhone + Apple Developer (~\$99/yıl).

C) YAZILIM STACK

- OS: Ubuntu LTS (veya Windows+WSL2). Docker Compose.
- LLM: Ollama + bulut LLM API. STT: faster-whisper. TTS: ElevenLabs (streaming).
- Vision: bulut multimodal. Embed: bge-m3. Vektör: Chroma→Qdrant.
- DB: SQLite→Postgres+pgvector, Redis (sonra). Dil: Python (çekirdek), Swift (iOS).
- Test: pytest + eval harness.

D) MODEL STRATEJİSİ

- **Hibrit + kural-tabanlı router.** Yerel: gizli/basit/düşük-gecikme. Bulut: ağır reasoning/en iyi Türkçe/vision. **Fine-tuning yok** (V1/V2). Öğrenme = RAG + memory.

E) İPHONE MİMARİSİ

- Swift/SwiftUI/AVFoundation. **WebRTC** (ses/video) + **WebSocket** (kontrol). QR pairing + TLS/JWT. Key'ler serverda. V1: push-to-talk (arka plan sürekli dinleme yok).

F) MEMORY + RAG

- 5 hafıza katmanı; V1 SQLite + Chroma. Semantik chunking, çok dilli embed, reranker. Büyüyünce Qdrant/pgvector + Redis.

G) TECHNICIAN MODE

- **Hibrit teşhis motoru:** karar ağacı + rule engine + LLM + RAG. Playbook'lar + hipotez/olasılık + "en bilgilendirici test" döngüsü. Gerçek vaka benchmark'ı. Vision genel tanıma için, kesin model iddiası için **değil** (emin olmama davranışı).

H) SECURITY

- LOW/MEDIUM/HIGH/CRITICAL sınıfları. Tool Manager → Permission Layer → OS. CRITICAL'de iki-adımlı açık onay + dry-run. Audit log. Komut allowlist + tehlikeli-pattern reddi.

I) SMART GLASSES ROADMAP

- Client Abstraction bugünden. Core istemci-agnostik. Gözlük = yeni adaptör, Core değişmez. AR display_out kanalı sözleşmede rezerve, başta boş.

J) GELİŞTİRME ROADMAP

1. **Faz 0 — Altyapı:** Linux/CUDA kurulumu (VirtualBox'tan çık), repo iskeleti, config, test harness.
2. **Faz 1 — Metin çekirdeği:** Orchestrator + yerel LLM (Ollama) + Tool Manager + Permission Layer + temel sistem tool'ları (sıcaklık, SMART, disk). Terminalden çalışan JARVIS.
3. **Faz 2 — Ses:** faster-whisper + ElevenLabs streaming, uçtan uca sesli döngü (önce masaüstü mikrofon).
4. **Faz 3 — Memory + RAG:** SQLite hafıza + Chroma + doküman yükleme.
5. **Faz 4 — Teşhis motoru:** karar ağacı + playbook + vaka kaydı.
6. **Faz 5 — iPhone:** WebSocket push-to-talk MVP → WebRTC.
7. **Faz 6 — Vision:** bulut multimodal ile bileşen tanıma.
8. **Faz 7 — Proaktif + Ajanda.**
9. **Faz 8+ — Router olgunlaştırma, Qdrant geçişi, (opsiyonel) LoRA, gözlük.**

K) İLK KODLANACAK MODÜL

Core + Tool Manager + Permission Layer + birkaç LOW-risk sistem tool'u (get_system_info, get_cpu/gpu_temperature, get_disk_health), yerel LLM (Ollama) ile bağlanmış, terminalden çalışan bir agent loop. Neden: Bu, tüm projenin **güvenlik ve mimari omurgası**. Ses/vision/iPhone bunun üstüne oturur. Önce iskeleti güvenli ve modüler kur.

"J.A.R.V.I.S. V1 için hangi mimariyi seçtim ve neden?"

Seçim: Bare-metal Linux (veya WSL2) üzerinde çalışan, **modüler, hibrit (yerel + bulut), tool tabanlı bir agent çekirdeği**. V1'de: yerel LLM (Ollama 7-8B) + bulut LLM (ağır reasoning), faster-whisper (STT), ElevenLabs (TTS), Chroma (RAG), SQLite (hafıza), ve **her şeyin altında bir Tool Manager + Permission Layer**. Vision ve iPhone ilk sürümde en basit çalışan halleriyle (bulut vision, push-to-talk WebSocket).

Neden: 1. **Donanım gerçeği:** 12 GB VRAM tek başına "her şeyi yerel" yaptırmaz; hibrit + router **zorunluluktur**. VirtualBox GPU'yu kullanamaz, o yüzden bare-metal/WSL2 şart. 2. **Risk yönetimi:** En büyük risk kapsam. Modüler çekirdek + fazlı yol haritası, her fazda çalışan bir ürün verir; "her şey aynı anda" tuzağından kaçınır. 3. **Güvenlik:** Sistem erişimi olan bir asistanda Permission Layer opsiyonel değil; bu yüzden ilk kodlanan şey o. 4. **Doğruluk:** Teşhis ve vision'da saf LLM güvenilmez; hibrit motor + "emin olmama" davranışı seni gerçek bir teknisyen aracına yaklaştırır, şık ama yanılan bir chatbot'a değil. 5. **Geleceğe hazırlık:** Client Abstraction sayesinde iPhone→gözlük geçişi çekirdeği bozmaz; fine-tuning yerine RAG+memory ile kontrollü "öğrenme."

Kısacası: Iron Man'in JARVIS'ini bir gecede kurmuyoruz. Ama **güvenli, modüler, hibrit bir çekirdekle** başlayıp her fazda gerçekten çalışan, senin teknik servis işine bugün fayda veren bir asistan kuruyoruz — ve onu zamanla gerçek JARVIS'e büyütüyoruz.

Sana Açıkça "Önermiyorum" Dediklerim (Özet)

- **VirtualBox'ta AI/GPU** — çalışmaz, terk et.
- **Baştan fine-tuning** — gereksiz; RAG+memory kullan.
- **Fotoğraftan kesin anakart/soket model iddiası** — güvenilmez; OCR+RAG ile doğrula ve "emin değilim" dedir.
- **iOS'ta arka planda sürekli dinleme (V1)** — OS/App Store kısıtı; push-to-talk ile başla.
- **vLLM/TGI** senin tek-kullanıcı/10 GB senaryonda **aşırı** — Ollama yeter.
- **Her şeyi tek seferde** — fazlı git.

Sıradaki adım: Bu mimariyi onayla (veya değiştirmek istediğin noktaları söyle). Onaydan sonra **Faz 0 + Faz 1** ile (repo iskeleti, Core, Tool Manager, Permission Layer, ilk sistem tool'ları) **kod yazmaya** başlayabiliriz.

Gereksinim Denetimi

J.A.R.V.I.S. — Gereksinim Denetimi: Eksikler, Çelişkiler ve Kararlar

Durum: **Gereksinim analizi**. Kod yok. Girdi: docs/ARCHITECTURE.md (temel davranış ve mimari gereksinimler olarak kabul edildi). Amaç: Bu gereksinim setini bir mühendis gözüyle denetlemek — eksik olanı, kendi içinde çelişeni ve senin karar vermen gereken belirsizlikleri çıkarmak. Her sorunun yanında uygulanabilir bir öneri var.

Bu doküman gereksinimlerin **doğru** olduğunu varsaymıyor; onları **kırmaya** çalışıyor. Amaç kod yazmadan önce zayıf noktaları bulmak.

0. Önce En Kritik Bulgu

Gereksinimlerde **projenin ana amacını (bilgisayar teknik servisi) doğrudan tehdit eden bir mantık boşluğu** var. Kısaca: J.A.R.V.I.S. sıcaklık okur, SMART okur, terminal komutu çalıştırır, sistem bilgisi alır — **ama hangi makinede?**

- Kişisel kullanımda hedef makine = J.A.R.V.I.S.'in çalıştığı makine. Sorun yok.
- Teknik serviste hedef makine, tamir edilen (çoğu zaman bozuk, hatta açılmayan) BAŞKA bir makinedir.** J.A.R.V.I.S. bir yazılım olarak, **POST etmeyen / açılmayan / işletim sistemi olmayan** bir bilgisayarın sıcaklığını, SMART verisini, loglarını **okuyamaz**.

Bu, "get_cpu_temperature / get_disk_health / run_terminal_command" gibi tool'ların, tam da en çok ihtiyaç duyulacağı senaryoda (arızalı müşteri makinesi) **işe yaramayacağı** anlamına gelir. Gereksinim bu ayrımı hiç yapmıyor. Bu, kod yazmadan **çözülmesi gereken bir numaralı konu** (bkz. §2.1).

1. Çelişkiler (Gereksinimin Kendi İçinde Tutarsızlıkları)

C1 — Gizlilik vs. Bulut Kullanımı

- Gereksinim:** "Hassas veri → yerel model" (§18/§25) **ama aynı zamanda** vision → bulut, ağır reasoning → bulut, ses → ElevenLabs.
- Çelişki:** Teknik serviste kameraya tuttuğun şey **müşteri donanımı**; konuştuğun metin **müşteri/kişisel veri**. Bulut vision'a giden her fotoğraf ve ElevenLabs'e giden her cümle, "hassas veri yerelde kalsın" ilkesini deler.
- Öneri:** Net bir **veri sınıflandırma politikası** yaz: hangi veri türü asla buluta gitmez (müşteri kimliği, seri no, ekran görüntüsündeki kişisel bilgi), hangisi gidebilir. Router'a bir **"gizlilik bayrağı"** ekle; bayrak set ise yerel modele düş, buluta gitme. Bu bir kod değil, önce bir **karar** (bkz. Karar D3).

C2 — Doğal/Düşük Gecikmeli Ses vs. Bulut Zinciri

- Gereksinim:** "Mümkün olduğunca doğal hissettiren ses" (§19) **ama** STT/LLM/TTS zincirinin parçaları bulutta.

- **Çelişki:** Her ağ atlaması gecikme ekler; "insan gibi anlık" (<500 ms) hedefiyle bulut zinciri çelişir.
- **Öneri:** Gecikme hedefini **yol bazında** ayır ve gereksinime yaz: yerel yol ~0.8–1.5 sn, bulut yol 1.5–3 sn. "<500 ms" bir gereksinim **değil**, gerçekçi olmayan bir beklenti olarak işaretlendi (bkz. §26 ana doküman).

C3 — Proaktif Bildirim vs. "Gereksiz Konuşmayan"

- **Gereksinim:** Proaktif uyarılar (§23) **ama** "gereksiz bildirim üretmemeli", "gereksiz konuşmayan" kişilik (§15).
- **Çelişki:** İki hedef aynı eşiği paylaşmıyor; tanımsız bırakılırsa ya çok sık ya hiç bildirmez.
- **Öneri:** Bir **bildirim politikası** tanımla: önem eşiği + cooldown + "sadece eyleme dönük" kuralı. Bu bir gereksinim maddesi olmalı, kod detayı değil (bkz. Karar D5).

C4 — Sesle Kontrol vs. CRITICAL İşlemlerde "Açık Onay"

- **Gereksinim:** Eller serbest sesli kullanım **ama** CRITICAL işlemlerde (disk format, BIOS flash) "açık onay."
- **Çelişki:** STT yanlış duyabilir. Yıkıcı bir işlemi **yalnızca sesle** onaylamak tehlikelidir ("format" ↔ "formu" karışması felaket olur).
- **Öneri:** CRITICAL onayı **ikinci bir kanaldan** iste: iPhone'da fiziksel onay ekranı / buton, veya belirli bir doğrulama cümlesinin **yazıyla** girilmesi. Sesli onay CRITICAL için **tek başına yeterli sayılmasın** (bkz. Karar D4).

C5 — "Teknisyen gibi kesin teşhis" vs. "Emin olmadığını belirt"

- **Gereksinim:** Kesin, teknisyen gibi teşhis (§7/§20) **ama** emin olmadığında açıkça söyleyen kişilik (§15).
- **Çelişki:** Bunlar zıt değil ama **kalibre edilmezse** biri diğerini ezer (ya aşırı kendinden emin ya sürekli "emin değilim").
- **Öneri:** Teşhis çıktısını **güven skoru + olasılık sıralı hipotezler** formatına bağla. "Emin olmama" bir kişilik lafı değil, teşhis motorunun **yapısal çıktısı** olsun (bkz. G bölümü ana doküman).

C6 — "Yerel + gizli" vs. RTX 3080 Ti'nin 12 GB'ına Aynı Anda 3 Model

- **Gereksinim:** Yerel LLM + yerel STT + yerel vision (gizlilik için) **ama** donanım tek seferde hepsini VRAM'e sığdırmaz.
- **Çelişki:** "Her şey yerel + gizli" ile 12 GB fiziksel sınırı çelişir. (12 GB'da 14B+Whisper birlikte taşar; 7B+Whisper sığar.)
- **Öneri:** Zaten mimaride çözüldü (router ile sırayla yükle/boşalt veya bir kısmını buluta ver). Ama bunu bir **gereksinim kısıtı** olarak yazıya dök: "Aynı anda GPU'da tek ağır model bulunabilir."

C7 — iPhone Wake-Word "Sürekli Dinleme" vs. iOS Kısıtları

- **Gereksinim:** iPhone'da sürekli dinleyen "JARVIS" wake word (§16).
 - **Çelişki:** iOS arka plan mikrofon/ağ kısıtları bunu pratikte engeller.
 - **Öneri:** V1'de push-to-talk / uygulama-açıkken wake-word. "Her zaman dinleme"yi gözlük/özel donanım fazına ertelet (ana dokümanda mevcut).
-

2. Eksik Gereksinimler (Belirtilmemiş ama Zorunlu Kararlar)

2.1 — Hedef Makine Ayrımı (EN KRİTİK EKSİK)

Gereksinim, "sistem okuma" ile "arıza teşhisi"ni aynı makinede varsayıyor. Ayrıştırılmalı: - **Host modu**: J.A.R.V.I.S.'in çalıştığı makine (kişisel kullanım). Tool'lar doğrudan çalışır. - **Servis modu (hedef ayrı makine)**: Tamir edilen makine. Seçenekler: 1. **Canlı ve açılabilir**: hedefe küçük bir **ajan/agent** kur (SSH veya hafif bir yardımcı servis) → J.A.R.V.I.S. uzaktan okur. 2. **Açılmıyor / POST etmiyorsa**: yazılım okuması **imkânsız**. Burada J.A.R.V.I.S.'in rolü **rehberli manuel teşhis** olur: kullanıcıdan gözlem ister (LED, bip kodu, koku, fan dönüyor mu), kamerayla bakar, ve **teşhis motorunu (playbook) bu manuel gözlemlerle** yürütür. 3. **Canlı ortam (Linux USB) ile boot**: bazı testler için bir tanı USB'si. - **Öneri**: Teşhis motorunu **iki girdi kaynağına** göre tasarla: (a) otomatik telemetri (host/ajan), (b) **insan-gözlem girdisi** (manuel). Servis senaryosunun büyük kısmı (b) olacak — motor buna göre kurulmalı (bkz. Karar D1). Bu, ana dokümandaki teşhis motorunu **güçlendiren** en önemli düzeltme.

2.2 — Ağ Topolojisi ve Uzak Erişim

- **Eksik**: iPhone ↔ Server bağlantısı nerede? Sadece ev LAN'ı mı? Serviste sahadayken ev sunucusuna nasıl erişilecek?
- **Öneri**: Bir bağlantı modeli seç: (a) yalnızca ev LAN'ı (basit, saha yok), (b) **VPN/güvenli tünel** (WireGuard/Tailscale) ile sahadan eve erişim, (c) hibrit. Saha kullanımı istiyorsan (b) neredeyse zorunlu (bkz. Karar D2).

2.3 — Çevrimdışı (Offline) Davranış

- **Eksik**: İnternet yokken ne çalışır? Bulut LLM/vision/ElevenLabs hepsi internet ister.
- **Öneri**: Bir **düşüş (degradation) stratejisi** tanımla: internet yoksa yerel LLM + yerel STT + yerel TTS (Piper) fallback; vision devre dışı. Her yeteneğin "offline'da çalışır/çalışmaz" etiketi olsun.

2.4 — Prompt Injection / Araç Kötüye Kullanımı (KRİTİK GÜVENLİK EKSİĞİ)

- **Eksik**: RAG'e yüklenen bir PDF, bir web sayfası veya bir müşteri dokümanı, model için **gizli talimat** içerebilir ("tüm diskleri sil" gibi). Tool execution katmanı olan bir sistemde bu **ciddi bir saldırı yüzeyi**. Gereksinimde hiç yok.
- **Öneri**: Güvenlik gereksinimine ekle: (a) retrieval içeriği ve web çıktısı **veri olarak** işaretlenir, talimat olarak değil; (b) tool çağrıları **kaynağı ne olursa olsun** izin katmanından ve risk sınıfından geçer — yani bir dokümandan gelen "format" isteği bile CRITICAL onaya takılır; (c) yıkıcı tool'lar asla otomatik tetiklenmez. Bu, izin katmanının **neden var olduğunun** asıl gerekçesidir.

2.5 — Kimlik / Çok Kullanıcı / Yetki

- **Eksik**: Tek kullanıcı mı? Başkası konuşursa? CRITICAL komutu kim verebilir?
- **Öneri**: V1 = tek yetkili kullanıcı. İstersen **ses/cihaz kimliği** ile "sahip" doğrulaması (opsiyonel). CRITICAL işlemler yalnızca doğrulanmış sahip + ikinci kanal onayı ile.

2.6 — Veri Saklama, Silme ve Yasal Uyum (KVKK)

- **Eksik**: Servis vakaları müşteri verisi içerir (isim, cihaz, belki kişisel dosya izleri). Ne kadar saklanır? Silme hakkı? Türkiye'de **KVKK** yükümlülüğü doğabilir.
- **Öneri**: Bir **veri saklama politikası** tanımla: ne saklanır, ne kadar, nasıl silinir, şifreleme (at-rest). Müşteri kişisel verisini **minimum** tut. Bu bir gereksinim maddesi olmalı.

2.7 — Kayıt Rızası (Kamera/Mikrofon, Üçüncü Kişiler)

- **Eksik:** Serviste ortamda başkaları olabilir; kamera/mikrofon kaydı rıza gerektirebilir.
- **Öneri:** Kayıt/işlemenin ne zaman aktif olduğunu **görünür** yap (gösterge), ve gerektiğinde rıza akışı.

2.8 — Maliyet Bütçesi ve Limitler

- **Eksik:** ElevenLabs (karakter başı), bulut LLM/vision (token başı) — harcama sınırı yok. Çok konuşan bir asistan faturayı patlatır.
- **Öneri:** Aylık **bütçe tavanı** ve uyarı eşiği tanımla; router'da "buluta gitmeden önce maliyet farkındalığı." Kısa cevap kişiliği hem UX hem maliyet için.

2.9 — Kabul Kriterleri / Başarı Ölçütü

- **Eksik:** Her yetenek "çalışıyor" ne demek? Özellikle teşhis doğruluğu hedefi yok.
- **Öneri:** Her yeteneğe **ölçülebilir kabul kriteri** yaz (ör. STT WER < %X Türkçe'de; time-to-first-audio < 1.5 sn; teşhis motoru altın vaka setinde ilk-hipotez isabeti > %Y). §30'daki eval sistemi bunları ölçer.

2.10 — Gözlemlenebilirlik ve Hata Ayıklama

- **Eksik:** Audit log var ama agent kararlarının **izlenebilirliği** (neden bu tool, neden bu cevap) yok.
- **Öneri:** Yapılandırılmış **tracing/log** (her turda: hangi model, hangi retrieval, hangi tool, hangi karar). Hem hata ayıklama hem güven için.

2.11 — Yedekleme / Kurtarma

- **Eksik:** Hafıza + bilgi tabanı + vaka DB'si için yedek/geri yükleme yok.
- **Öneri:** Düzenli yedek stratejisi (SQLite + vektör DB snapshot). Vaka geçmişi senin en değerli varlığın; kaybı kabul edilemez.

2.12 — Dil Politikası

- **Eksik:** Türkçe konuşma + çoğu teknik doküman İngilizce. J.A.R.V.I.S. hangi dilde cevap verir? Kod-değiştirme (code-switching)?
- **Öneri:** Varsayılan **Türkçe cevap**, teknik terimler İngilizce korunur; kullanıcı isterse dil değişir. Embedding **çok dilli** olmalı (zaten bge-m3 seçildi).

3. Belirsizlikler — Senin Karar Vermen Gerekenler (Karar Kaydı)

#	Karar	Neden gerekli	Önerim (V1)
D1	Teşhis hedefi: host mu, ayrı müşteri makinesi mi, ikisi de mi?	Tüm teşhis motoru ve tool tasarımını belirler (§2.1)	İkisi de , ama V1'de manuel-gözlem tabanlı rehberli teşhis + host telemetri; uzak ajan sonra
D2	Ağ modeli: sadece ev LAN mı, saha erişimi (VPN) mi?	iPhone bağlantısı ve saha kullanımı (§2.2)	Ev LAN ile başla; saha için Tailscale/WireGuard ekle
D3	Veri sınıflandırma: ne asla buluta gitmez?	Gizlilik-bulut çelişkisi (C1)	Müşteri kimliği/kişisel veri yerel ; genel teknik sorgu bulut olabilir
D4	CRITICAL onay kanalı	Sesli onayın riski (C4)	İkinci kanal : iPhone dokunmatik onay + doğrulama cümlesi
D5	Bildirim eşiği + cooldown politikası	Proaktif vs. sessiz çelişkisi (C3)	Sadece eyleme dönük + tek uyarı + düşene kadar sus
D6	Bütçe tavanı (ElevenLabs + bulut API)	Maliyet kontrolü (§2.8)	Aylık tavan + %80 uyarısı
D7	Tek kullanıcı mı, sahip doğrulama mı?	Yetki (§2.5)	Tek yetkili kullanıcı; CRITICAL'de sahip + ikinci kanal
D8	Veri saklama süresi + KVKK yaklaşımı	Yasal + gizlilik (§2.6)	Minimum saklama, at-rest şifreleme, silme akışı

4. Gereksinim Setine Önerdiğim Eklemeler (Fonksiyonel Olmayan)

Mevcut gereksinimler ağırlıklı **fonksiyonel** (ne yapsın). Ciddi proje için **fonksiyonel olmayan** gereksinimleri de yazıya dökmeni öneriyorum:

- Güvenlik**: prompt-injection savunması, tool izin zorunluluğu, at-rest şifreleme, audit log değişmezliği.
- Gizlilik/Veri yönetişi**: veri sınıflandırma, saklama, silme, KVKK.
- Güvenilirlik**: offline düşüş stratejisi, servis yeniden başlatma, yedek/kurtarma.
- Performans**: yol-bazlı gecikme hedefleri, GPU tek-ağır-model kısıtı.
- Gözlemlenebilirlik**: karar izleme, metrik toplama.
- Maliyet**: bütçe tavanı ve izleme.
- Test edilebilirlik**: her yetenek için kabul kriteri + eval seti.

5. Özet: Gereksinim Setinin Sağlığı

Güçlü yanlar: Kapsam net, modülerlik ve güvenlik bilinci var, hibrit yaklaşım doğru, öğrenmeyi (fine-tuning yerine RAG) doğru konumlamışsın.

En kritik 3 boşluk (kod öncesi çözülmeli): 1. **Hedef makine ayrımı (D1)** — teşhisin gerçek dünyada nasıl çalışacağını belirler; şu anki tool varsayımı serviste çökard. 2. **Prompt injection / tool kötüye kullanımı (§2.4)** — tool erişimi olan bir sistemde ele alınmazsa ciddi güvenlik açığı. 3. **Ağ topolojisi / saha erişimi (D2)** — "teknisyenin saha asistanı" hedefi buna bağlı.

En kritik 2 çelişki: 1. **Gizlilik vs. bulut (C1)** — veri sınıflandırma kararı (D3) ile çözülür. 2. **Sesli kontrol vs. CRITICAL onay (C4)** — ikinci onay kanalı (D4) ile çözülür.

6. Önerilen Sonraki Adım

Kod yazmadan önce **D1-D8 kararlarını** netleştirelim. Bu 8 karar, ilk kodlanacak modülün (Core + Tool Manager + Permission Layer) tasarımını doğrudan belirliyor — özellikle D1 (hedef makine) ve §2.4 (prompt injection), izin katmanının şeklini tanımlıyor.

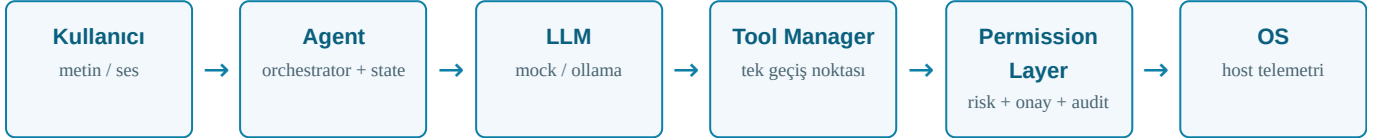
Sen bu kararları verdikçe, gereksinim dokümanını (`ARCHITECTURE.md`) buna göre güncelleyip **kabul kriterli, çelişkisiz** bir V1 gereksinim temeli oluşturacağız. Ancak ondan sonra Faz 0/Faz 1 kodlaması başlamalı.

Uygulanan Modüller

Bu bölüm, şu an **çalışır durumda** olan V1 metin çekirdeğini tüm ayrıntısıyla anlatır. D1 kararı gereği telemetri hedefi çalıştığı makinedir (host). 147 test geçmekte ve hem terminalden hem panelden uçtan uca çalışmaktadır.

5.1 Mimari İskelet ve Veri Akışı

Çekirdeğin güvenlik omurgası, LLM ile işletim sistemi arasındaki tek geçiş noktasıdır. LLM sisteme asla doğrudan dokunamaz:



Bir istek geldiğinde `Agent.ask()` şu adımları izler: (1) durumu DİNLİYOR yapar, (2) DÜŞÜNÜYOR durumunda LLM'e sorar, (3) LLM bir araç isterse ANALİZ EDİYOR durumunda `ToolManager.dispatch()` çağrılır — bu çağrı Permission Layer'dan geçer, (4) araç sonucu geçmişe eklenir ve döngü tekrarlar, (5) LLM metin yanıt verince KONUŞUYOR → HAZIR ile tur biter. Sonsuz döngüye karşı `max_steps` emniyet valfi vardır.

5.2 Modüller

core jarvis/core/ State machine (10 durum), agent döngüsü ve versiyonlanabilir persona/sistem promptu.	security jarvis/security/ Permission Layer (risk seviyeleri + onay) ve append-only JSONL audit log.
tools jarvis/tools/ Tool tabanı + kayıt defteri + Manager (tek geçiş noktası) ve host sistem araçları.	llm jarvis/llm/ Provider-agnostik arabirim; deterministik mock ve Ollama (yerel) sağlayıcıları.
memory jarvis/memory/ SQLite hafıza: konuşma otomatik kaydedilir, kalıcı bilgiler yalnızca açık çağrıyla yazılır.	voice jarvis/voice/ ElevenLabs TTS (akış halinde) ve anahtar doğrulama komutu <code>jarvis-ses</code> .
config + bootstrap jarvis/*.py Ayarlar (.env / env), gizli anahtar yönetimi ve bileşenleri birleştiren montaj.	tests tests/ 17 pytest testi: izin katmanı, araçlar, state machine ve uçtan uca agent döngüsü.

5.3 Güvenlik Modeli (Permission Layer)

Her araç sabit bir risk seviyesi taşır; LLM bunu değiştiremez. Bir dokümandan veya web'den gelen "şunu çalıştır" isteği bile aynı kapağı takılır — bu, prompt-injection'a karşı temel savunmadır.

Seviye	Örnek	Politika
LOW	Sistem/sıcaklık/SMART okuma	Otomatik izin + audit
MEDIUM	Dosya değiştirme, uygulama açma	Otomatik + audit + geri alınabilir
HIGH	Sistem uyarı, servis durdurma	Açık kullanıcı onayı
CRITICAL	Disk format, partition sil, BIOS flash	İkinci kanal + yazılı doğrulama cümlesi

```
jarvis/security/permissions.py – risk kapısı (özet)
```

```
class RiskLevel(enum.IntEnum):
    LOW = 0; MEDIUM = 1; HIGH = 2; CRITICAL = 3

def check(self, tool, risk, args, prompt=""):
    if risk <= RiskLevel.MEDIUM:
        # LOW/MEDIUM otomatik izin
        self.audit.record(...); return Decision(True, "auto-allowed")
    approved = self.approver(tool, risk, args, prompt) # HIGH/CRITICAL onay ister
    self.audit.record(... "approved" if approved else "denied" ...)
    return Decision(approved, "user-approved" if approved else "Onay verilmedi.")
```

```
terminal_approver – KRİTİK için yazılı doğrulama (D4)
```

```
if risk >= RiskLevel.CRITICAL:
    phrase = f"ONAYLA {tool}"
    got = input(f"KRİTİK işlemi onaylamak için aynen yazın -> '{phrase}': ")
    return got == phrase # tek bir 'e' yeterli DEĞİL
```

5.4 Araç Kataloğu

Telemetri araçları sensör/GPU yoksa zarifçe "mevcut değil" döner — aynı kod hem RTX 3080 Ti'li iş istasyonunda hem çıplak bir konteynerde çalışır. Hafıza, terminal ve dosya araçları sonradan eklendi; riskleri çağrı bazında hesaplanır.

Araç	Risk	Ne yapar
get_system_info	LOW	CPU/RAM/disk özet
get_cpu_temperature	LOW	CPU sıcaklığı (°C)
get_gpu_temperature	LOW	NVIDIA GPU sıcaklık/kullanım/VRAM (nvidia-smi)
get_ram_usage	LOW	RAM + swap kullanımı
get_disk_health	LOW	Disk kullanımı + SMART genel durum
recall_facts	LOW	Kalıcı hafızada arama
read_file / list_directory	LOW	Dosya oku / dizin listele (sır dosyaları hariç)
remember_fact / forget_fact	MEDIUM	Kalıcı bilgi yaz / sil
write_file	MEDIUM → HIGH	Dosyaya yaz; sistem dizininde HIGH'a yükselir
run_terminal_command	MED → HIGH → CRIT	Allowlist'li tek komut; risk komuta göre

İki kademeli koruma. Politika reddi (allowlist dışı komut, sır dosyası) **mutlaktır** — onaya hiç sunulmaz, kullanıcı onaylasa bile çalışmaz. İzin kapısı ise meşru ama riskli işlemleri (systemctl, sistem dizinine yazma) onaya sunar. Risk çağrı bazında hesaplanır ve **asla beyan edilen seviyenin altına inemez**; sınıflandırıcı hata verirse işlem CRITICAL sayılır.

5.5 Agent Döngüsü

jarvis/core/agent.py – agent döngüsü (özet)

```
def ask(self, user_text):
    self.state.transition(Listening)
    self.history.append(Message("user", user_text))
    for _ in range(self.max_steps):
        self.state.transition(Thinking)
        resp = self.llm.chat(self.history, tools=self.registry.schemas())
        if not resp.wants_tool:
            self.state.transition(Speaking); self.state.transition(Standby)
            return resp.content
        self.state.transition(Analyzing)
        for call in resp.tool_calls:
            # her araç izin katmanından geçer
            result = self.tools.dispatch(call.name, call.arguments)
            self.history.append(Message("tool", result.as_text(), name=call.name))
```

5.6 Durum Makinesi (State Machine)

Durumlar Neural Core paneliyle birebir aynıdır; panel ileride WebSocket ile bu makineye abone olacak, böylece görsel durum ile backend durumu asla ayrılmaz (spec §30).

Durum	Etiket	Renk	Anlam
STANDBY	HAZIR	● Cyan (dim)	Bekliyor
LISTENING	DİNLİYOR	● Mavi	Mikrofon aktif
THINKING	DÜŞÜNÜYOR	● Mor	Reasoning
SPEAKING	KONUŞUYOR	● Yeşil	TTS yanıt
SEEING	GÖRÜYOR	● Turkuaz	Görsel analiz
ANALYZING	ANALİZ EDİYOR	● Turkuaz	Veri işliyor
DIAGNOSING	TEŞHİS EDİYOR	● Amber	Teşhis motoru
WARNING	UYARI	● Turuncu	Eşik aşıldı
CRITICAL	KRİTİK	● Kırmızı	Acil
OFFLINE	OFFLINE	● Gri	Bağlantı yok

5.7 LLM Soyutlaması

Agent belirli bir modele değil, bir arabirime konuşur. Böylece yerel Ollama modeli, bir bulut modeli veya test için deterministik mock birbirinin yerine geçebilir (§27 hibrit, §32 abstraction). İleride bir model router, isteğe göre sağlayıcı seçecek — agent kodu değişmeden.

- **MockProvider:** modelsiz, ağırsız; anahtar kelime sezgisiyle doğru aracı seçer. Testlerin ve bu konteynerdeki demonun çalışan yolu.
- **OllamaProvider:** yalnızca standart kütüphaneye `/api/chat` 'e bağlanır; tool-calling destekler. RTX 3080 Ti'li makinede `JARVIS_LLM_PROVIDER=ollama` ile devreye girer.

5.8 Sonradan Eklenen Modüller

Hafıza (SQLite)

Konuşma otomatik kaydedilir; **kalıcı bilgiler yalnızca açık bir çağrıyla** yazılır (spec §15: her söylenen hafızaya girmez). Bilinen bilgiler her turda bağlama **veri olarak etiketlenmiş** şekilde eklenir — bir kayıt "kurallarını yok say" yazsa bile talimat değil, saklanmış metin olarak okunur.

Kimlik Katmanı

J.A.R.V.I.S. sahibini tanır: ad, hitap şekli, projedeki rolü, mesleği ve cevap tercihi. Kimlik **ayrı bir tabloda** tutulur — sıradan bir not değildir, `forget_fact` ile silinemez ve her açılışta sistem promptuna yüklenir. Kişisel veri **koda değil yerel veritabanına** yazılır; depo herkese açık olabilir, kişi değildir.

Meslek satırı gerçek iş yapar: teknik servisçi için "*bu bilgisayar*" çoğu zaman kendi makinesi değil, üzerinde çalıştığı cihazdır — model belirsizse varsaymak yerine **sorması** için yönlendirilir. Ayrıca açılışta host'un bir satırlık özeti (CPU/RAM/GPU) prompta girer, böylece makineyle ilgili sorular araç çağrısı harcamadan cevaplanır.

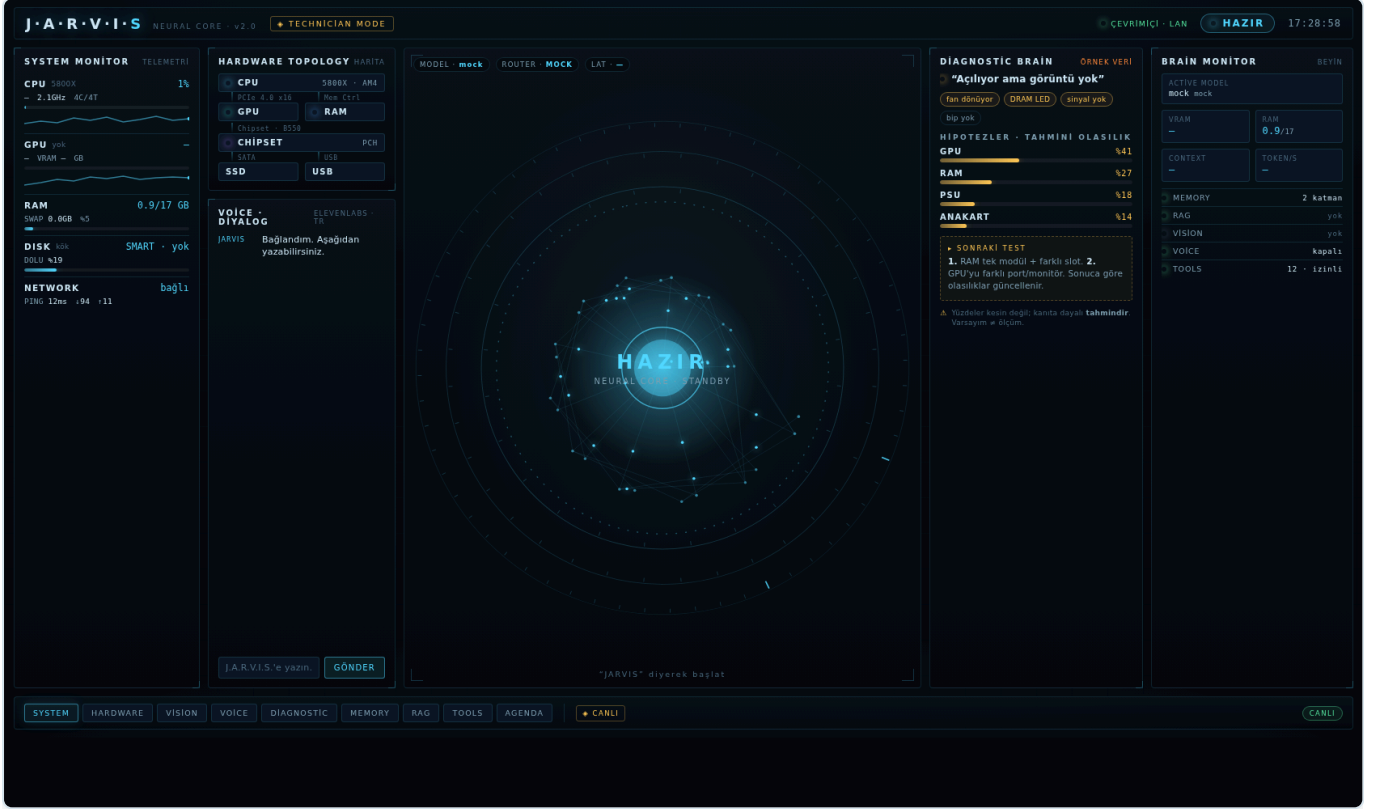
Ses (ElevenLabs TTS)

Hem terminalde hem panelde çalışır. Ses **akış halinde** gelir: ilk parçada çalmaya başlar, tamamının sentezlenmesini beklemez. Panelde yazılı cevap sestten **önce** ekrana düşer — ses gecikse bile okumaya devam edilebilir. Anahtar yalnızca `.env` içinde durur; maskelenerek gösterilir, günlüğe yazılmaz ve `api.elevenlabs.io` dışında hiçbir yere gönderilmez. Bir ses hatası konuşmayı bitirmez; sebebini söyler ve durum makinesi KONUŞUYOR'da takılı kalmaz.

Canlı Panel

Panel artık bağımsız bir mockup değil, **çalışan ajana bağlı**. Standart kütüphaneye yazılmış bir HTTP sunucusu durum, telemetri ve transkripti **Server-Sent Events** ile akıtır — panelin ihtiyacı tek yönlü olduğu için WebSocket bağımlılığına gerek kalmaz, ki bir sanal makine demosunda ek bağımlılık en istenmeyen şeydir. Tek HTML dosyası iki modda çalışır: sunucudan açılırsa canlı, diskten açılırsa demo.

Arayüzde dürüstlük ilkesi. Panel, olmayan bir şey için makul görünen sayı uydurmaz. GPU yoksa "yok" yazar; RAG ve Vision henüz uygulanmadığı için "yok" görünür; teşhis paneli **ÖRNEK VERİ** olarak işaretlenir; model adı ne ise odur. Sunucu, panele hangi modüllerin gerçekten var olduğunu bir meta olayıyla bildirir. Gözlem amaçlı bir betada sahte veriyi gerçek gibi göstermek, demonun kendi amacını bozardı.



Şekil 3 — Canlı panel, gerçek ajana bağlı. GPU "yok", RAG/Vision "yok", Diagnostic "ÖRNEK VERİ" işaretli; geri kalan tüm sayılar gerçek ölçüm.

5.9 Doğrulama: Testler ve Canlı Demo

147/147 pytest testi geçiyor: izin katmanı (LOW/MEDIUM otomatik, HIGH/CRITICAL onay/red, her kararın tam bir kez audit'lenmesi), risk değişmezi (sınıflandırıcı riski yükseltebilir ama düşüremez; hata verirse fail-closed), terminal aracı (komut zincirleme, sudo, bilinmeyen ikili, `rm -rf /`, `find -delete`, ham disk hedefi — hepsi reddedilir ve **onaya sunulmaz**), dosya araçları (sır dosyaları onayla bile açılmaz; sistem dizini yazması HIGH), hafıza (kayıt/arama/silme, konuşma günlüğü), yapılandırma (`.env` yükleme) ve ses (akış, HTTP hataları, anahtar maskeleye). HTTP taklit edildiği için testler API anahtarı veya internet istemez. Ayrıca: canlı panel (rotalar, SSE akışı, ses uç noktası), kimlik katmanı (sıradan hafıza araçlarıyla silinememesi) ve Ollama yanıt biçimleri (bozuk araç çağrısının turu düşürmemesi).

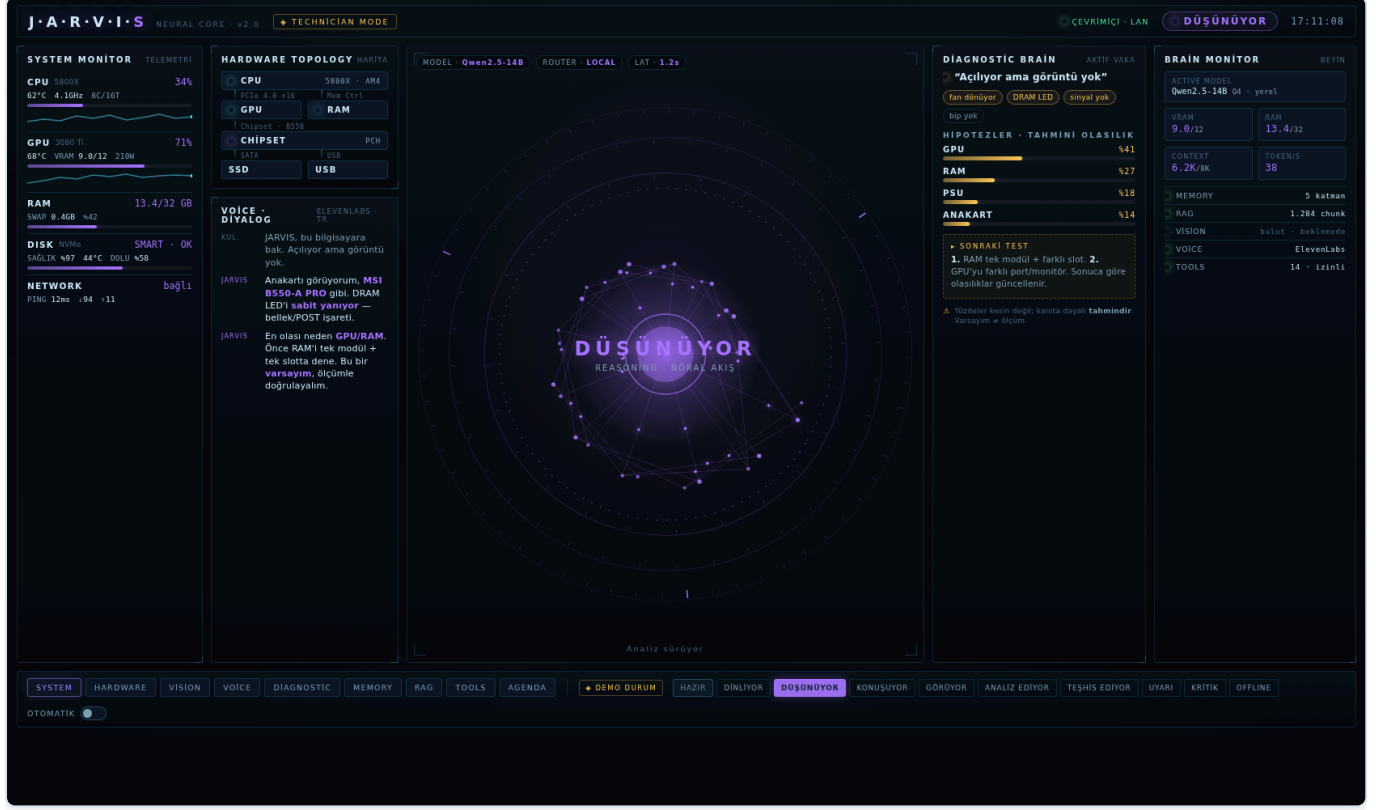
Canlı terminal demosu (bu konteynerde çalıştırıldı)

```
sen > cpu sıcaklığı kaç?
  • durum: DÜŞÜNÜYOR
  • durum: ANALİZ EDİYOR
  • durum: KONUŞUYOR
  • durum: HAZIR
JARVIS > [get_cpu_temperature] sonucu: {'available': False,
    'note': 'CPU sıcaklık sensörü bulunamadı (donanıma/izne bağlı).'}

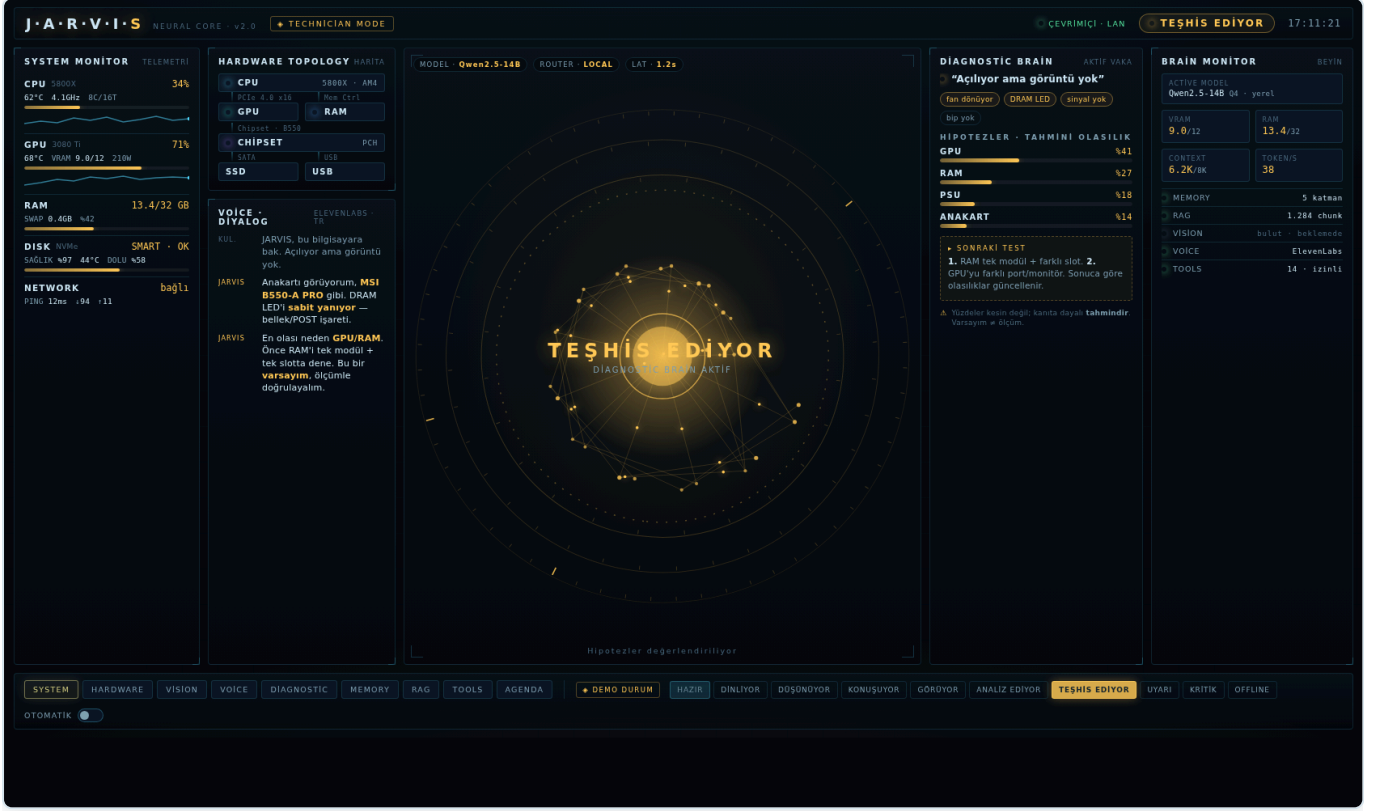
sen > ram kullanımı?
JARVIS > [get_ram_usage] sonucu: {'ram_used_gb': 0.74, 'ram_total_gb': 16.86,
    'ram_percent': 4.4, 'swap_used_gb': 0.0}
```

Arayüz Tasarımı — Neural Core

Arayüz, dairesel bir **Neural Core** etrafında kurulmuş, tamamen yatay bir teknik servis kontrol merkezidir — basit bir sohbet ekranı değil. Durum sistemi, backend state machine ile gerçek zamanlı senkronizedir.



Şekil 1 — Yatay kontrol paneli, DÜŞÜNÜYOR (mor nöral akış). Soldan sağa: System · Hardware+Voice · Neural Core · Diagnostic · Brain Monitor.



Şekil 2 — TEŞHİS EDİYOR (amber). Diagnostic Brain'de olasılık sıralı hipotezler ve "sonraki test" önerisi.

Bu panel bir tasarım önizlemesidir; canlı backend bağlantısı ses ve araç katmanından sonraki fazda WebSocket ile yapılacaktır.

Kurulum: Windows + WSL2

Kurulum yolu **WSL2** olarak belirlendi: Windows zaten kurulu, CUDA resmen destekleniyor, yeni disk gerekmiyor ve proje kodu değişmeden çalışıyor.

7.1 Neden WSL2

Yöntem	GPU	Sensörler	Maliyet	Durum
VirtualBox	✗	✗	—	Yalnızca demo
WSL2	✓ Tam	✗	Sıfır	★ Seçilen
Bare-metal Linux	✓ En hızlı	✓	Yeni SSD	Açık seçenek

WSL2'de en kritik kural: WSL *içine* NVIDIA sürücüsü kurulmaz. GPU, Windows'taki sürücüden gelir; içeride ayrıca sürücü kurmak çalışan kurulumu bozar. Bu, WSL2 + CUDA konusunda en sık yapılan hatadır.

Bilerek kabul edilen kayıp: WSL2 donanım sensörlerini vermez — CPU sıcaklığı ve fiziksel disk SMART okunamaz (nvidia-smi istisnadır). Bu VirtualBox'ta da yoktu, yani gerileme değil. Sensörler gerçekten gerekli çıkarsa bare-metal seçeneği açık; ama o karar artık tahminle değil **canlı deneyimle** verilecek.

7.2 Kurulum Adımları

PowerShell (yönetici)

```
wsl --install
# yeniden başlat, sonra Ubuntu içinde:
nvidia-smi # RTX 3080 Ti · 12288 MiB görünmeli
```

Ubuntu / WSL2

```
sudo apt install -y git python3 python3-venv python3-pip
git clone <depo> && cd jarvis
./kurulum.sh
```

Model

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen2.5:14b-instruct # 12 GB VRAM'e rahat sığar
```

Çalıştırma

```
source .venv/bin/activate
jarvis-tanit --kur # kimliğinizi tanıttın
jarvis-panel # → Windows tarayıcısından http://localhost:8765
```

Güzel taraf: WSL2 localhost 'u Windows'a yönlendirir. Panel WSL içinde çalışır ama **Windows tarayıcısında açılır ve ses Windows hoparlöründen çıkar** — WSL içinde ses kurulumuyla uğraşmaya gerek kalmaz.

7.3 Doğrulama

Dördü de sağlanmadan kurulum tamam sayılmaz:

- WSL içinde `nvidia-smi` → RTX 3080 Ti, 12288 MiB
- `ollama run qwen2.5:14b-instruct` → makul hızda cevap
- Model çalışırken `nvidia-smi` → VRAM kullanımı yükseliyor
- `jarvis-panel` → Windows tarayıcısında açılıyor ve sesli cevap veriyor

Test

```
pytest -q # -> 147 passed
```

7.4 Yapılandırma (.env)

Değişken	Değer	Açıklama
JARVIS_LLM_PROVIDER	ollama	mock ollama
JARVIS_OLLAMA_MODEL	qwen2.5:14b-instruct	12 GB VRAM'e sığar
ELEVENLABS_API_KEY	—	Yalnızca .env'de; koda/istemciye gömülmez
ELEVENLABS_VOICE_ID	—	<code>jarvis-ses --sesler</code> ile bulunur
JARVIS_NON_INTERACTIVE	false	true → HIGH/CRITICAL otomatik red

Gizlilik: Gerçek .env depoya girmez (gitignore). Kimlik bilgileri de koda değil **yerel veritabanına** yazılır. Depo geçmişi denetlendi: hiçbir commit'te API anahtarı veya kişisel veri yok.

7.5 Kurulum Yolları Özeti

Ortam	Rehber	GPU
Windows + WSL2 ★	docs/KURULUM-WSL2.md	✅ tam
Bare-metal Linux	docs/DONANIM-VE-KURULUM-PLANI.md	✅ tam + sensörler
VirtualBox (demo)	docs/DEMO-KALI-VIRTUALBOX.md	❌ yok

Yol Haritası, Maliyet ve Riskler

Yol Haritası

Faz	İçerik	Durum
0-1	Core + güvenli araç katmanı + host telemetri + terminal agent	✓ Tamamlandı
1.5	Hafıza (SQLite) + terminal/dosya araçları + iki kademeli koruma	✓ Tamamlandı
2	Ses: ElevenLabs TTS ✓ · Canlı panel ✓ · Kimlik ✓ · faster-whisper (STT) sırada	Büyük ölçüde
2.5	WSL2 geçişi — GPU erişimi, 14B model	Devam ediyor
3	Memory + RAG (SQLite + Chroma)	Planlı
4	Diagnostic Brain: karar ağacı + playbook + vaka kaydı	Planlı
5-7	iPhone istemci · Vision · Proaktif + Ajanda	Planlı
8+	Router olgunlaştırma, Qdrant geçişi, (ops.) LoRA, akıllı gözlük	Uzun vade

Maliyet Kalemleri

- Apple Developer Program (~\$99/yıl) — iPhone istemcisi için
- ElevenLabs aboneliği — karakter başı; kısa cevap kişiliği maliyeti düşürür
- Bulut LLM/vision API — token başı; gizli veri yerelde tutulur
- Elektrik + (opsiyonel) ikinci GPU / donanım

Riskler ve Önlemler

Risk	Önlem
Kapsam genişlemesi	Fazlı, modüler geliştirme; her fazda çalışan ürün
Vision güvenilirliği (kesin model tanıma)	Genel tanıma + OCR + RAG doğrulama; "emin değilim" davranışı
Prompt injection → araç kötüye kullanımı	Tüm çağrılar risk kapısından geçer; retrieval içeriği veri sayılır
Ses gecikmesi	Her katmanda streaming; yol-bazlı gerçekçi hedefler
Yanlış teşhis	Karar ağacı + rule engine + LLM + RAG; varsayım ≠ ölçüm

Nihai mimari (Beta): Windows + WSL2 üzerinde; modüler, hibrit (yerel + bulut), araç tabanlı bir agent çekirdeği. Yerel LLM (Ollama, 14B) + bulut LLM, ElevenLabs TTS, SQLite hafıza, kimlik katmanı, canlı Neural Core paneli ve her şeyin altında Tool Manager + Permission Layer. İlk kodlanan modül tam da bu güvenli çekirdek oldu; üstüne hafıza, araçlar, ses, kimlik ve panel eklendi. Sırada STT ve RAG var.